Polymarket US Trading Engine: Rebuild & Adversarial Review Walkthrough

1. Overview & Strategy Summary

The Polymarket bot has been rebuilt from the ground up for **Polymarket US** (CFTC-regulated, legal for US retail, official `polymarket-us` SDK v2.4.1) under branch `pmus-rebuild`.

The Core Strategy

Retail capital (~$70 bankroll) cannot compete in median 0.1-cent spreads or high-frequency orderbook battles on major sports events. Instead, the bot generates steady yield by targeting **Polymarket US's Liquidity Incentive Program (LIP)** rewards: - **Passive Maker Quoting**: Posts resting post-only limit orders (`participateDontInitiate=True`) at qualifying price levels within active incentive windows. - **Greedy Wing Allocation**: Evaluates candidate pools and concentrates capital into completing the 5,000-contract `target_size` on cheap qualifying wings ($0.010-$0.015 bids or $0.985-$0.990 asks). A $47 commitment at $0.010 captures up to 90%+ of the reward pool. - **Event Diversification**: Strictly quotes at most 1 market per sporting event to eliminate correlated multi-line exposure. - **Pre-Event Cancellation**: Automatically cancels all resting orders 300 seconds before match start time to prevent adverse selection during live play.

---

2. Adversarial Review & Applied Fixes

Following `/interrogate` multi-model review, the following critical issues were identified, implemented, and verified:

Subsystem	Issue Identified	Resolution Implemented	File
Market Selector	`max_spread` pools disqualified all single-sided orders because `estimate_lip_score` requires both sides to qualify.	Tested counter-book depth; added joint `both` candidate evaluation when budget permits so two-sided wings qualify.	`pmus/strategy/market_selector.py`
Order State Sync	`LiveEngine` had disconnected WebSocket stubs and no REST fill sync, causing ghost orders in SQLite ledger.	Added active REST open order and fill reconciliation on each engine cycle, marking missing orders as FILLED or CANCELED.	`pmus/venue.py`, `pmus/engine.py`
Pre-Trade Risk Scope	`check_pre_submit` was called per-market with local orders, blinding the risk gate to total portfolio collateral.	Aggregated global open orders across all active markets from ledger before validating against `$60` budget and `$70` collateral cap.	`pmus/risk.py`, `pmus/engine.py`
Margin Misclassification	`SELL_LONG` exit orders were treated as shorting (`col = 1 - px`), consuming buying power to exit positions.	Corrected required collateral for `SELL_LONG` to `Decimal("0")`.	`pmus/risk.py`
Realized Loss Stop	`LiveEngine.run` passed hardcoded	Implemented `Ledger.get_realized_loss_today(now)`	`pmus/ledger.py`, `pmus/engine.py`

Subsystem	Issue Identified	Resolution Implemented	File
	`realized_loss_today=0`, bypassing the `$10.00` daily stop.	computing net closed fill P&L since UTC midnight.	
Order Deadlock	New quote collateral was checked before old orders were canceled on the exchange, causing false buying power rejections.	Executed `to_cancel` orders before checking buying power and submitting `to_create` batches.	`pmus/engine.py`
Saturated Books	Quoter fell through to deploy full budget when book already met `target_size`, taking risk for $0 LIP score.	Suppressed quotes when `shortfall <= 0`, and constrained book depth summation to levels within `max_spread`.	`pmus/strategy/lip_quoter.py`
Basket Scanner	Treated multi-line sports spreads (-10.5, -7.5) as mutually exclusive, emitting phantom $3 arbitrage signals.	Enforced `market.mutually_exclusive == True` on all event markets before evaluating basket arbitrage.	`pmus/strategy/basket_scanner.py`
Deployment & Ops	Kill switch was outside Docker mounts; `pmus.service` hardcoded `--mode shadow` overriding `/etc/pmus.env`.	Updated kill switch to check `settings.data_dir / "STOP"`; removed `--mode shadow` from systemd unit; enabled SQLite WAL mode.	`pmus/risk.py`, `deploy/pmus.service`, `pmus/ledger.py`

3. Verification & Pass Receipts

A. Unit Test Suite (100% Green)

Command:

```
.\venv\Scripts\python.exe -m pytest pmus/tests -v
```

Output:

```
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\neststock\scripts\polymarket_bot
collected 81 items

pmus/tests/test_basket_scanner.py (5 tests) PASSED
pmus/tests/test_engine.py (3 tests) PASSED
pmus/tests/test_fees.py (17 tests) PASSED
pmus/tests/test_ledger.py (4 tests) PASSED
pmus/tests/test_lip_quoter.py (10 tests) PASSED
pmus/tests/test_live_and_risk.py (15 tests) PASSED
pmus/tests/test_market_selector.py (7 tests) PASSED
pmus/tests/test_models.py (3 tests) PASSED
pmus/tests/test_report.py (2 tests) PASSED
pmus/tests/test_risk.py (5 tests) PASSED
```

```
pmus/tests/test_shadow.py (8 tests) PASSED
pmus/tests/test_venue.py (4 tests) PASSED

============================== 81 passed in 0.43s ==============================
```

B. Live Public Gateway Probe

Command:

```
.\venv\Scripts\python.exe -m pmus probe
```

Output:

```
Fetching active markets...
Sampled active markets count: 20
  Market: tec-mlb-nlchamp-2026-09-27-lad | Cat: sports | Tick: 0.001 | MinQty: 1
  Market: tec-mlb-champ-2026-09-27-cws | Cat: sports | Tick: 0.001 | MinQty: 1
  Market: tec-mlb-nlchamp-2026-09-27-mil | Cat: sports | Tick: 0.001 | MinQty: 1

Fetching active incentive programs...
Sampled incentive windows count: 200
  Window: asc-eurolg-asv-czv-2026-10-13-neg-10pt5 | Pool: $5 | Discount: 0.35 | Target: 5000 | MaxSpread: None
  Window: asc-eurolg-asv-czv-2026-10-13-neg-10pt5 | Pool: $20 | Discount: 0.25 | Target: 5000 | MaxSpread: None

Fetching order book for asc-eurolg-asv-czv-2026-10-13-neg-10pt5...
  Book for asc-eurolg-asv-czv-2026-10-13-neg-10pt5: bids=1, asks=1, spread=0.9800
  Best Bid: px=0.0100, qty=300.0000
  Best Ask: px=0.9900, qty=325.0000
```

C. Live Performance Report (`pmus report --since 1h`)

```
============================================================
  POLYMARKET US PERFORMANCE REPORT (SINCE 1H)
============================================================
  Total Fills:             0
  Total Trade Volume:      $0.00
  Maker Rebates Earned:    $0.0000
  Taker Fees Paid:         $0.0000
  Realized P&L:            $0.0000
------------------------------------------------------------
  LIP Snapshots Recorded:  871
  LIP Qualifying Seconds:  855 (Bid: 307, Ask: 0)
  Estimated LIP Accrued:   $0.0319 (estimate)
------------------------------------------------------------
  LIP MARKET BREAKDOWN:
  - Market asc-eurolg-asv-czv-2026-10-13-neg-10pt5 (live):
    Pool=$20.00, Dur=334800s (93.0h), Share=92.8%, Est=$0.0022
    Completion Cost=$46.75, Remainder Est=$6.9775, Projected Total=$6.9797
    [ELIGIBLE FOR PAYOUT: > $1.00 THRESHOLD]
------------------------------------------------------------
  ESTIMATED NET EARNINGS:  $0.0319
============================================================
```

---

4. Deployment Runbook for AWS Server

Prerequisites on AWS Host (Ubuntu 22.04 LTS / 24.04 LTS)

1. **Clone repository**: `bash git clone <repo_url> /home/ubuntu/polymarket_bot cd /home/ubuntu/polymarket_bot git checkout pmus-rebuild`
2. **Setup virtual environment**: `bash python3 -m venv venv ./venv/bin/pip install --upgrade pip ./venv/bin/pip install -r pmus/requirements.txt`
3. **Environment configuration** (`/etc/pmus.env`): `bash sudo nano /etc/pmus.env` Insert credentials and operational settings: `ini PMUS_KEY_ID=your_polymarket_us_key_id PMUS_SECRET_KEY=your_polymarket_us_secret_key PMUS_MODE=shadow PMUS_TOTAL_BUDGET=60.00 PMUS_MAX_COLLATERAL=70.00 PMUS_MAX_COLLATERAL_PER_MARKET=30.00 PMUS_MAX_MARKETS=4 PMUS_DAILY_LOSS_STOP=10.00`
4. **Install and start systemd service**: `bash sudo cp deploy/pmus.service /etc/systemd/system/pmus.service sudo systemctl daemon-reload sudo systemctl enable pmus sudo systemctl start pmus`
5. **Inspect logs**: `bash sudo journalctl -u pmus -f`

Safe Go-Live Gate

1. Run in `PMUS_MODE=shadow` for **48–72 hours** on the server.
2. Check performance using `./venv/bin/python -m pmus report --since 48h` .
3. Verify that:
4. Zero unexpected exceptions occurred.
5. Accrued LIP estimates exceed the $1.00 payout floor per market-period.
6. Projected net return is positive.
7. When ready to transition to live trading, update `/etc/pmus.env` to `PMUS_MODE=live` and restart the service: `bash sudo systemctl restart pmus`
8. **Emergency Kill Switch**: To instantly cancel all resting orders and pause trading, create the stop file: `bash touch /home/ubuntu/polymarket_bot/pmus/data/STOP`